- **Course / Section:** ICS344 F11
- **Student Name(s) + ID(s):** Naba Al Antaif 2022782160
- **Your DVSA Website URL:** http://dvsa-website-test-053108159524-us-east-1.s3-website.us-east-1.amazonaws.com
- **AWS Region:** us-east-1
- **Date:** 27/4/2026
- **Vulnerability Title(s):** Over-Privileged Function (Lesson 7)

Part 1) Goal and Vulnerability Summary:

This lesson demonstrates the Over-Privileged Lambda Execution Role vulnerability, where the Lambda function DVSA-SEND-RECEIPT-EMAIL is assigned an IAM role with permissions far beyond what it requires.

The **main affected component** is the IAM execution role attached to the Lambda function.

Security impact: an attacker who gains access to the function's temporary credentials can perform unauthorized actions such as reading DynamoDB tables, listing S3 buckets, or sending arbitrary SES emails.

The **weakness** arises from broad wildcard permissions (s3:*, dynamodb:*, ses:*) that violate the principle of least privilege.

Part 2) Why This Works / Root Cause:

The vulnerability exists because the Lambda execution role is configured with excessive permissions that are not required for its intended purpose. Instead of granting only the minimal SES and CloudWatch Logs permissions, the role includes full administrative access to multiple AWS services. The root cause is a failure to enforce least-privilege IAM policies.

Part 3) Environment and Setup:

- DVSA URL: **http://dvsa-website-test-053108159524-us-east-1.s3-website.us-east-1.amazonaws.com**

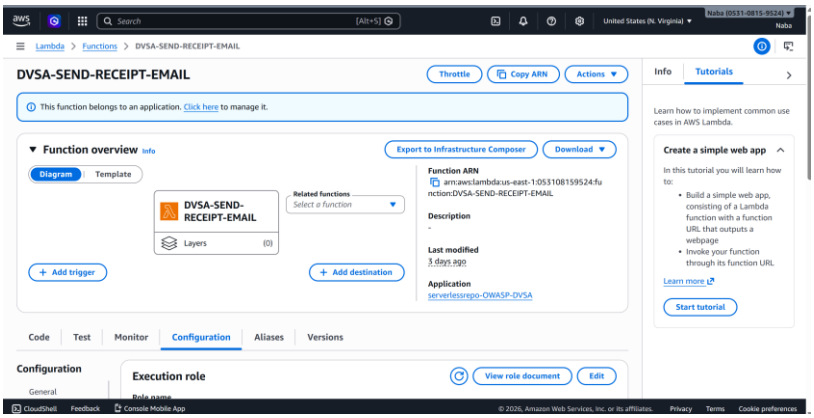
- AWS Services Involved:
  - Lambda Function: DVSA-SEND-RECEIPT-EMAIL
  - IAM Execution Role: Attached to the above Lambda
  - CloudWatch Log Group: /aws/lambda/DVSA-SEND-RECEIPT-EMAIL
  - S3 Bucket: dvsa-receipts-bucket-053108159524-us-east-1 (used to trigger the Lambda)

- Tools Used:
  - AWS Console (IAM, Lambda, CloudWatch, CloudTrail)
  - AWS CLI commands were executed inside Ubuntu (WSL) terminal on my local machine.
  - Browser: used to upload the receipt file and trigger the Lambda

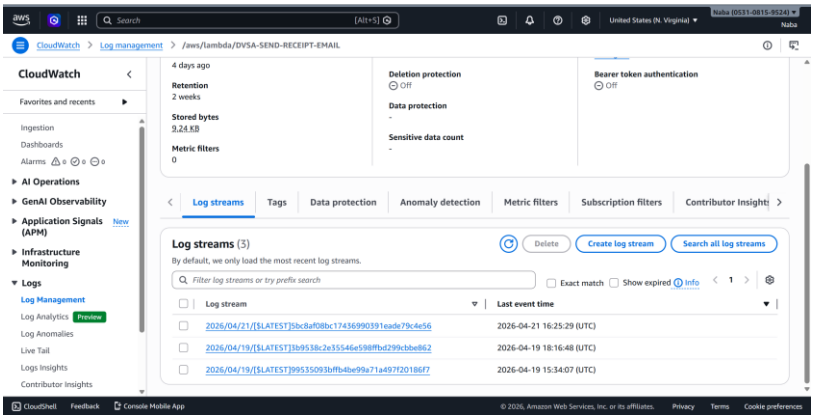
- User Context:
    - Normal DVSA user to trigger the Lambda
    - Attacker profile using temporary credentials printed in CloudWatch logs

Part 4) Reproduction Steps:

1. Navigate to the DVSA website and upload any file to trigger the DVSA-SEND-RECEIPT-EMAIL Lambda function.



2. Go to AWS Console → Lambda → DVSA-SEND-RECEIPT-EMAIL → Monitor → View logs in CloudWatch.

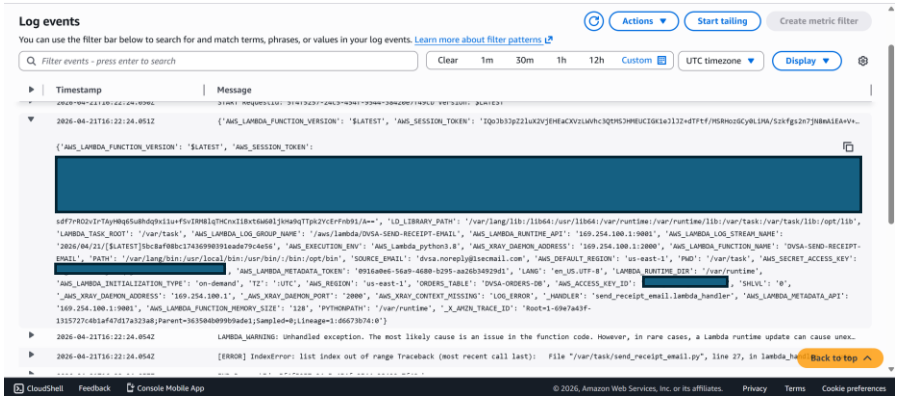


3. Open the most recent Log Stream.
4. Locate the printed Python dictionary containing environment variables.
5. Observe that the log includes sensitive fields such as:
    a. AWS_ACCESS_KEY_ID
    b. AWS_SECRET_ACCESS_KEY
    c. AWS_SESSION_TOKEN
6. Using the AWS CLI, attempt to perform unauthorized actions:
aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1

7. The command succeeds, proving that the Lambda role has excessive permissions.

Part 5) Evidence and Proof:

1. CloudWatch Log Evidence

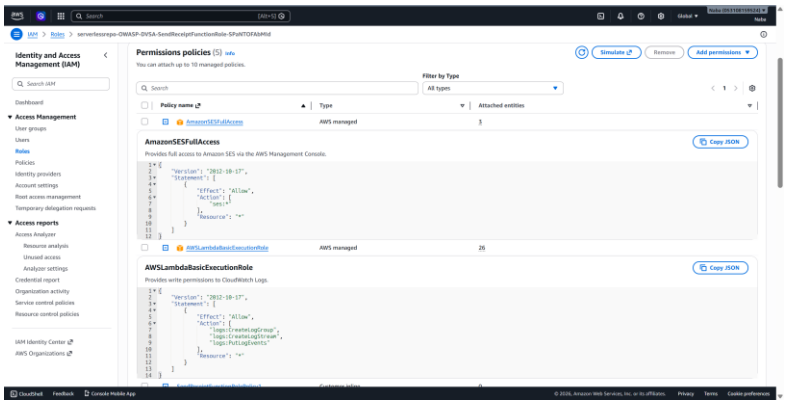


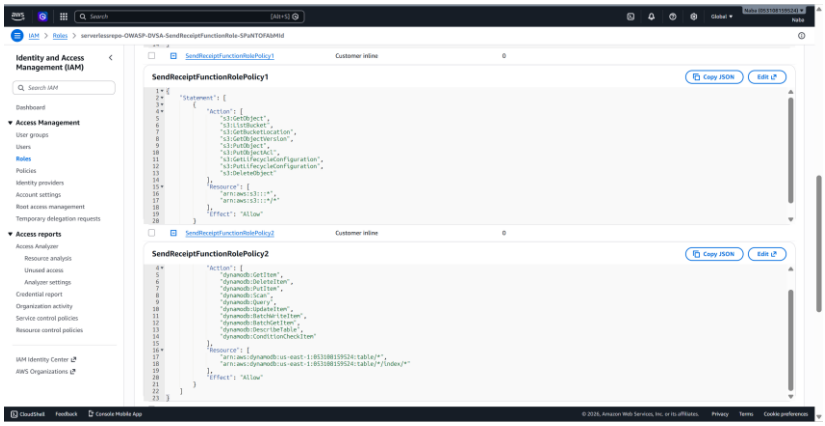
The dictionary includes temporary AWS credentials such as:

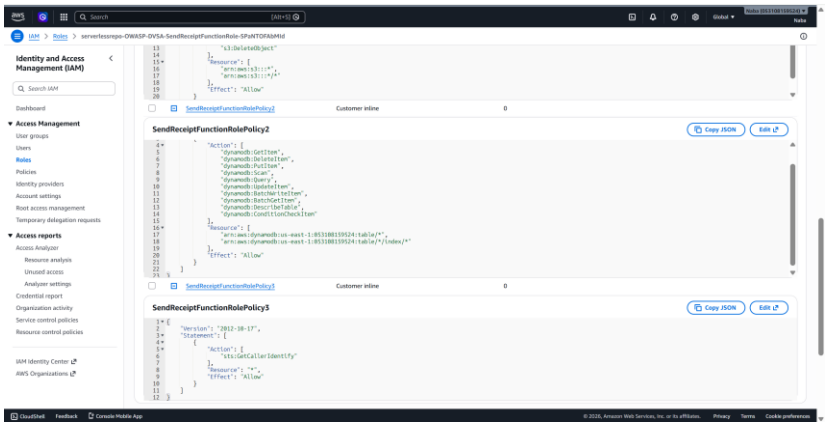
- AWS_SESSION_TOKEN
- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY

This demonstrates that the Lambda function leaked its temporary credentials in plaintext logs.

2. IAM Role Evidence







The IAM execution role attached to the Lambda function shows overly broad permissions, including wildcard actions such as: s3:*, dynamodb:*, ses:*

This confirms that the role violates the principle of least privilege.

3. AWS CLI Evidence (Unauthorized Access)

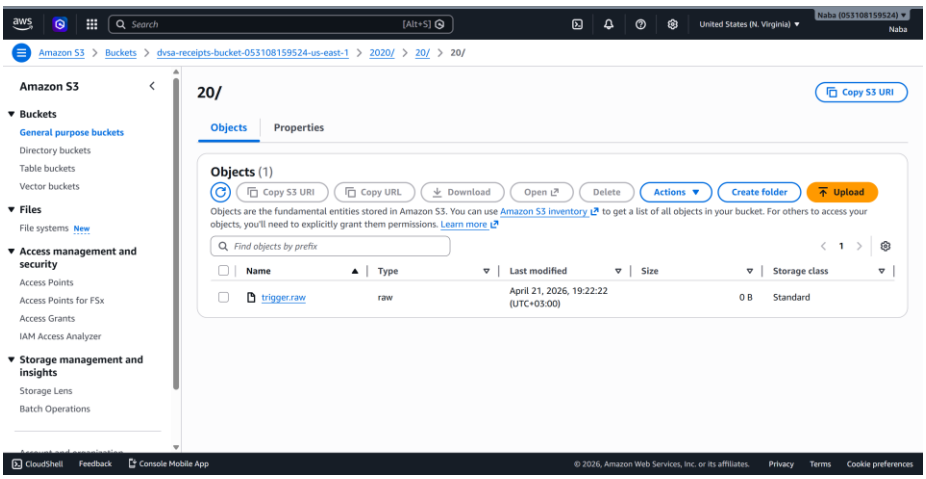
Ubuntu terminal:

```
naba_@LAPTOP-F5TDE2CQ: $ aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1
{
    "Items": [
        {
            "itemList": {
                "M": {
                    "1017": {
                        "N": "1"
                    }
                }
            },
            "totalAmount": {
                "N": "45"
            },
            "orderStatus": {
                "N": "300"
            },
            "confirmationToken": {
```

This proves that the Lambda's temporary credentials can be used to access unrelated AWS resources.

4. S3 Trigger Evidence

S3 bucket path shows the uploaded trigger.raw file under: 2020/20/20/trigger.raw



This confirms that the Lambda was triggered correctly during the test.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability is caused by an over-privileged IAM execution role attached to the DVSA-SEND-RECEIPT-EMAIL Lambda function.

To mitigate the issue, the role must be restricted to the minimum permissions required for the Lambda to perform its intended task.

Fix Strategy

- Remove all wildcard permissions such as s3:*, dynamodb:*, and ses:*.
- Grant only the specific SES permissions needed for sending emails:
  - ses:SendEmail
  - ses:SendRawEmail
- Keep only the required CloudWatch Logs permissions:
  - logs:CreateLogGroup
  - logs:CreateLogStream
  - logs:PutLogEvents
- Use CloudTrail's "Used permissions" feature to confirm which actions the Lambda actually performs.
- Apply the principle of least privilege to ensure the Lambda cannot access unrelated AWS services.

Part 7) Code / Config Changes

Before (Vulnerable IAM Policies)

1. AmazonSESFullAccess (AWS Managed Policy)

This policy grants full SES access:

AmazonSESFullAccess

Provides full access to Amazon SES via the AWS Management Console.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "ses:*"
            ],
            "Resource": "*"
        }
    ]
}
```

This allows the Lambda to send any SES email, manage identities, and perform administrative SES actions.

2. SendReceiptFunctionRololicy1 (Inline Policy — S3)

This policy grants broad S3 permissions across all buckets:

SendReceiptFunctionRolePolicy1

```
{
    "Statement": [
        {
            "Action": [
                "s3:GetObject",
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetObjectVersion",
                "s3:PutObject",
                "s3:PutObjectAcl",
                "s3:GetLifecycleConfiguration",
                "s3:PutLifecycleConfiguration",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::*",
                "arn:aws:s3:::*/*"
            ],
            "Effect": "Allow"
        }
```

This gives the Lambda the ability to read, write, and delete objects in any S3 bucket.

3. SendReceiptFunctionRolePolicy2 (Inline Policy — DynamoDB)

This policy grants full access to all DynamoDB tables in the account:

SendReceiptFunctionRolePolicy2

```
            "Action": [
                "dynamodb:GetItem",
                "dynamodb:DeleteItem",
                "dynamodb:PutItem",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:UpdateItem",
                "dynamodb:BatchWriteItem",
                "dynamodb:BatchGetItem",
                "dynamodb:DescribeTable",
                "dynamodb:ConditionCheckItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:053108159524:table/*",
                "arn:aws:dynamodb:us-east-1:053108159524:table/*/index/*"
            ],
            "Effect": "Allow"
        }
    ]
}
```

This allows the Lambda to read, modify, and delete data across all DynamoDB tables.

4. SendReceiptFunctionRolePolicy3 (Inline Policy — STS)

This policy allows identity inspection

SendReceiptFunctionRolePolicy3

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "sts:GetCallerIdentify"
            ],
            "Resource": "*",
            "Effect": "Allow"
        }
    ]
}
```

After (Least-Privilege IAM Policy)

To fix the vulnerability, all broad S3, DynamoDB, SES, and STS permissions must be removed.
The Lambda only needs:

- Permission to send emails using SES
- Permission to write logs to CloudWatch

AmazonSESFullAccess_update

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "ses:SendEmail",
            "Resource": "*"
        }
    ]
}
```

Deleted the original AmazonSESFullAccess and add new inline policy.

SendReceiptFunctionRolePolicy1

```
{
    "Statement": [
        {
            "Action": [
                "s3:GetObject",
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetObjectVersion",
                "s3:PutObject",
                "s3:PutObjectAcl",
                "s3:GetLifecycleConfiguration",
                "s3:PutLifecycleConfiguration",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::dvsa-receipts-bucket-053108159524-us-east-1/*"
            ],
            "Effect": "Allow"
        }
    ]
```

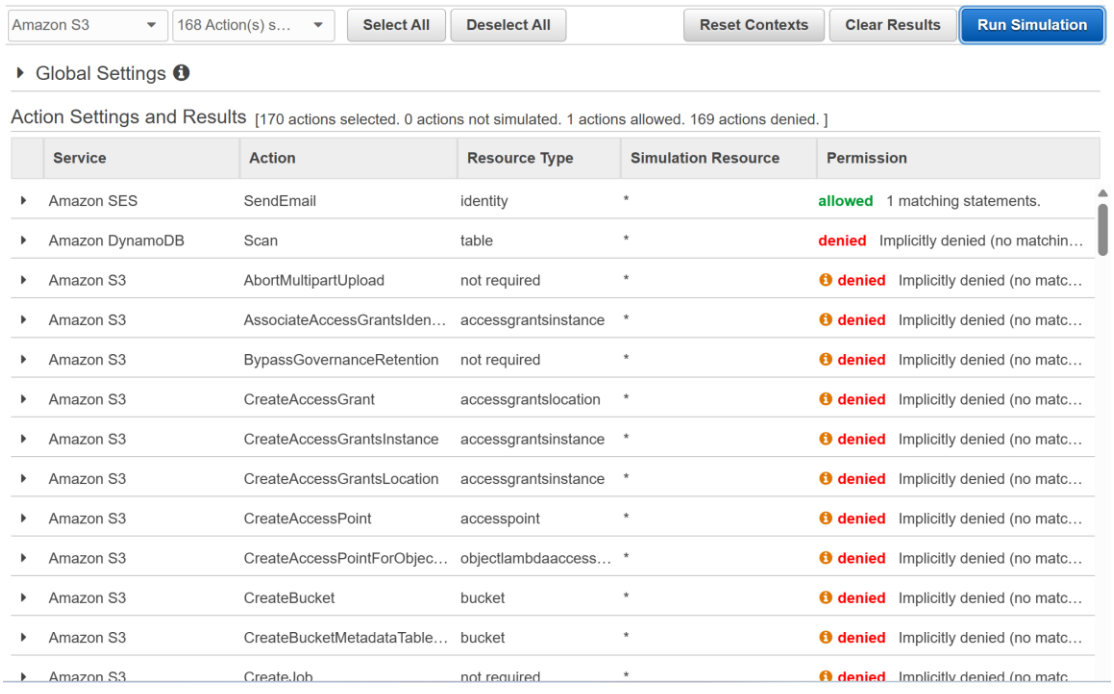
SendReceiptFunctionRolePolicy2

```
{
    "Statement": [
        {
            "Action": [
                "dynamodb:GetItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:053108159524:table/DVSA-ORDERS-DB"
            ],
            "Effect": "Allow"
        }
    ]
}
```

Policy3 only allows sts:GetCallerIdentity which is safe, so no need to change it.

Part 8) Verification After Fix

After applying the least-privilege IAM policy, the same exploitation steps were repeated to verify that the vulnerability is no longer exploitable. The expected behavior is that unauthorized operations should now be blocked, while legitimate DVSA functionality continues to work normally.




```
naba_@LAPTOP-F5TDE2CQ: $ aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1

aws: [ERROR]: An error occurred (AccessDeniedException) when calling the Scan operation: User: arn:aws:iam::053108159524:user/attacker is not author
ized to perform: dynamodb:Scan on resource: arn:aws:dynamodb:us-east-1:053108159524:table/DVSA-ORDERS-DB because no identity-based policy allows the
 dynamodb:Scan action
```

Part 9) Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged	Lambda must only send SES emails; must not access DynamoDB; temporary credentials must not enable privilege escalation	• CloudWatch logs • AM policies • AWS CLI output • Policy Simulator	Lambda sends SES email only; no DynamoDB access	Successful DynamoDB Scan using leaked credentials

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Over-Privileged	• Violates least-privilege • Expose sensitive data • Allows privilege escalation	Accidental Misconfiguration	IAM Role policy updated to SES-only permissions	• DynamoDB Scan now returns AccessDenied • Policy Simulator confirms restrictions

Part 10) Takeaway / Lessons Learned

In serverless environments, IAM roles define the security boundary, so any misconfiguration directly exposes backend data. The key lesson is that serverless functions must be granted only the exact permissions required, and nothing more. Applying least-privilege IAM policies and validating them with tools like IAM Policy Simulator prevents similar issues and ensures secure, predictable behavior.